

# Convolutional Neural Networks (CNNs)

CS480/680 - University of Waterloo  
Study Notes

## Lecture 12

### Contents

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                 | <b>3</b>  |
| <b>2</b>  | <b>Mathematical Foundation: Convolution</b>         | <b>3</b>  |
| 2.1       | Continuous Convolution . . . . .                    | 3         |
| 2.2       | Convolution as Smoothing . . . . .                  | 3         |
| 2.3       | Discrete Convolution . . . . .                      | 4         |
| 2.4       | 2D Convolution for Images . . . . .                 | 4         |
| <b>3</b>  | <b>Edge Detection Example</b>                       | <b>4</b>  |
| <b>4</b>  | <b>CNN Architecture Components</b>                  | <b>5</b>  |
| 4.1       | Neural Unit as Convolution . . . . .                | 5         |
| 4.2       | Gabor Filters . . . . .                             | 5         |
| <b>5</b>  | <b>Convolutional Layer Architecture</b>             | <b>5</b>  |
| 5.1       | 1D Convolution . . . . .                            | 5         |
| 5.2       | 2D Convolution . . . . .                            | 6         |
| <b>6</b>  | <b>Pooling Layers</b>                               | <b>6</b>  |
| 6.1       | Max Pooling Visualization . . . . .                 | 6         |
| <b>7</b>  | <b>Complete CNN Architecture: Digit Recognition</b> | <b>7</b>  |
| 7.1       | LeNet-style Architecture for MNIST . . . . .        | 7         |
| 7.2       | Architecture Details . . . . .                      | 7         |
| <b>8</b>  | <b>Key Benefits of CNNs</b>                         | <b>8</b>  |
| <b>9</b>  | <b>Convolutional Layer Hyperparameters</b>          | <b>8</b>  |
| 9.1       | Number of Filters . . . . .                         | 8         |
| 9.2       | Kernel Size (Filter Size) . . . . .                 | 8         |
| 9.3       | Stride . . . . .                                    | 8         |
| 9.4       | Padding . . . . .                                   | 9         |
| 9.5       | Output Size Calculation . . . . .                   | 9         |
| <b>10</b> | <b>Hyperparameter Examples</b>                      | <b>9</b>  |
| <b>11</b> | <b>Training CNNs</b>                                | <b>10</b> |
| 11.1      | Backpropagation with Weight Sharing . . . . .       | 10        |
| 11.2      | Optimization Algorithms . . . . .                   | 10        |
| <b>12</b> | <b>Important Insights</b>                           | <b>11</b> |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>13 Historical Context &amp; Applications</b> | <b>11</b> |
| <b>14 Summary</b>                               | <b>12</b> |

# 1 Introduction

## Key Concept

**Convolutional Neural Networks** have become the workhorse for image processing and computer vision tasks. They are designed to handle large-scale data with high dimensionality efficiently.

CNNs are a special class of neural networks with specific architectural properties that enable them to:

- Scale to high-dimensional data (e.g., images)
- Handle sequences of varying lengths
- Detect features automatically through learning
- Achieve translation invariance through weight sharing

## 2 Mathematical Foundation: Convolution

### 2.1 Continuous Convolution

**Definition 1** (Convolution). A convolution is an operation on two functions  $X$  and  $W$  that produces a third function  $Y$ . Mathematically, the convolution of  $X$  by  $W$  is defined as:

$$Y(t) = (X \star W)(t) = \int_{-\infty}^{\infty} X(\tau)W(t - \tau) d\tau \quad (1)$$

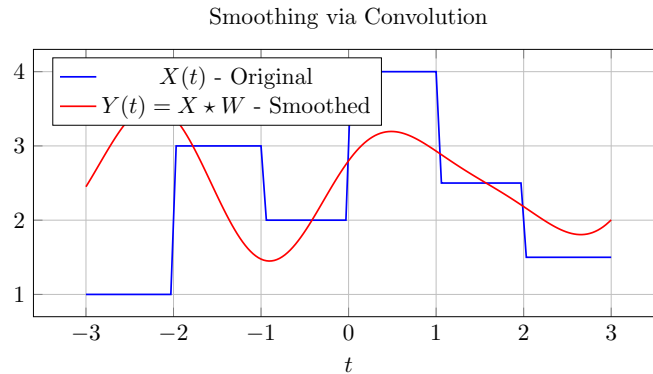
where  $\star$  denotes the convolution operator.

**Notation:** The star operator ( $\star$ ) denotes convolution, *not* multiplication.

### 2.2 Convolution as Smoothing

One important application of convolution is smoothing non-smooth functions. Consider a function  $X(t)$  with discontinuities that we want to smooth using a Gaussian weighting function:

$$W(\tau) = K \exp\left(-\frac{(t - \tau)^2}{2\sigma^2}\right) \quad (2)$$



**Interpretation:** The convolution computes  $Y$  as a weighted combination of different values of  $X$ , where weights are defined by  $W$ . With a Gaussian kernel, this acts as a weighted average over a neighborhood.

## 2.3 Discrete Convolution

For discrete sequences (as opposed to continuous functions), the convolution becomes a summation:

$$Y(i) = \sum_j X(j)W(i-j) \quad (3)$$

## 2.4 2D Convolution for Images

For images (2D discrete functions), convolution is defined over two dimensions:

$$Y(t_1, t_2) = \sum_i \sum_j X(i, j)W(t_1 - i, t_2 - j) \quad (4)$$

where:

- $(t_1, t_2)$  are the coordinates (x, y)
- $X(i, j)$  represents pixel intensity at position  $(i, j)$
- $W(i, j)$  represents weights applied to each pixel
- $Y(t_1, t_2)$  is the output image

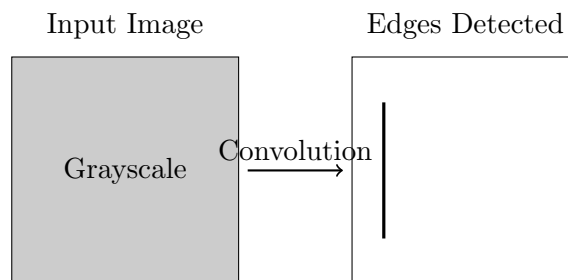
## 3 Edge Detection Example

**Example 1** (Vertical Edge Detection). *To detect vertical edges in a grayscale image, we can use a simple convolution filter that computes the difference between adjacent pixels:*

$$Y(i, j) = X(i, j) - X(i-1, j) \quad (5)$$

*This can be represented as a convolution with the filter:*

$$W = \begin{bmatrix} -1 & 1 & 0 \end{bmatrix} \quad (6)$$



$$\text{Filter: } W = \begin{bmatrix} -1 & 1 & 0 \end{bmatrix}$$

**Result:** Black edges appear when transitioning from white to black; white edges appear when transitioning from black to white.

## 4 CNN Architecture Components

### 4.1 Neural Unit as Convolution

In a neural network, a unit computes:

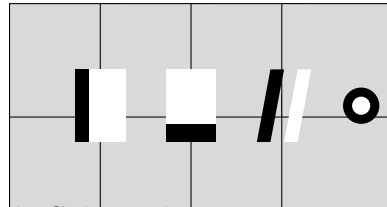
$$z = \sum_i w_i x_i \quad (\text{linear combination}) \quad (7)$$

$$y = h(z) \quad (\text{nonlinear activation}) \quad (8)$$

**Key Insight:** This linear combination can be interpreted as a convolution when applied to a local patch/window, with weights defining the filter.

### 4.2 Gabor Filters

Gabor filters are biologically-inspired filters that detect edges and textures at various orientations and scales:



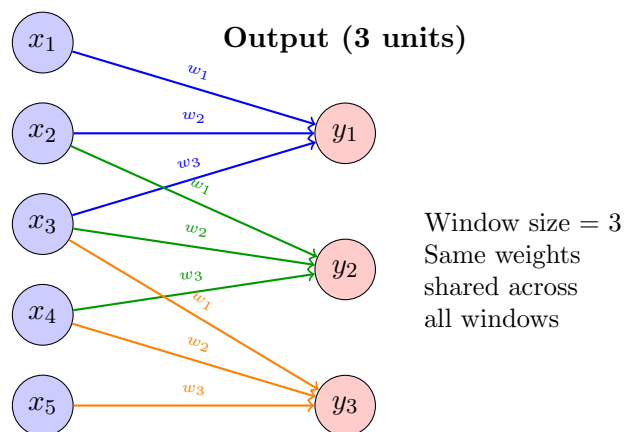
Example Gabor Filter Feature Maps  
White = positive weights, Black = negative weights, Gray = zero

**Note:** Refer to original lecture slides for detailed Gabor filter visualizations.

## 5 Convolutional Layer Architecture

### 5.1 1D Convolution

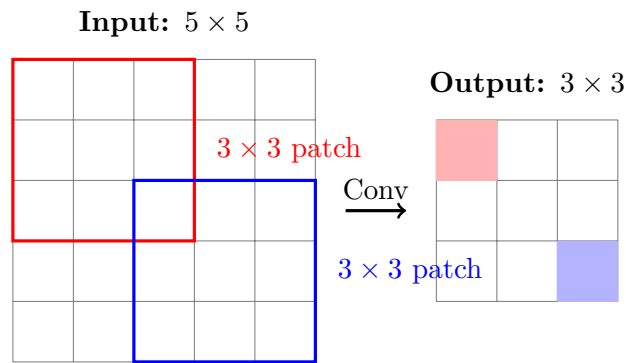
Input (5 units)



#### Key Properties:

- Each output connects to only 3 inputs (sparse connectivity)
- Same weights  $\{w_1, w_2, w_3\}$  reused for all outputs (weight sharing)
- Total parameters: only 3 weights (vs. 15 for fully connected)

## 5.2 2D Convolution



Filter (kernel):  $3 \times 3$  with weights  
 $w_1, w_2, \dots, w_9$  ap-  
 plied to each patch

### Process:

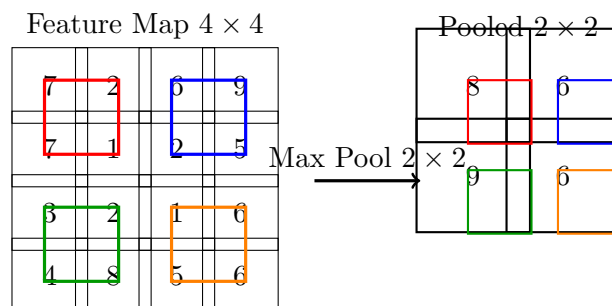
1. Apply  $3 \times 3$  filter to top-left patch  $\rightarrow$  output[0,0]
2. Slide filter across columns and rows
3. Same 9 weights reused for every patch
4. Total output positions:  $3 \times 3 = 9$  (for this example)

## 6 Pooling Layers

**Definition 2** (Pooling). *Pooling is a commutative operation that combines several units to form local invariance. Common pooling operations include:*

- *Max pooling:*  $\max(x_1, x_2, \dots, x_n)$
- *Average pooling:*  $\frac{1}{n} \sum_{i=1}^n x_i$
- *$L^2$  norm pooling:*  $\sqrt{\sum_{i=1}^n x_i^2}$

### 6.1 Max Pooling Visualization

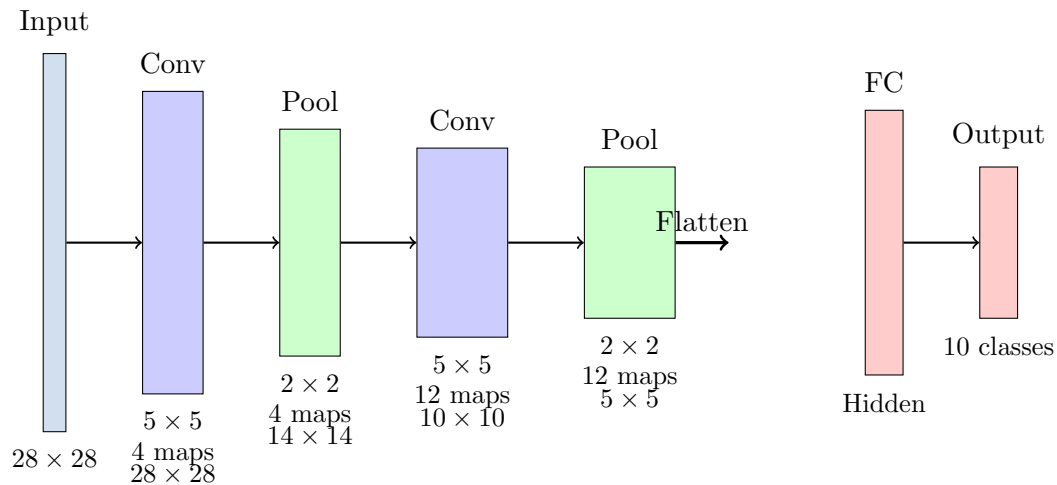


**Purpose:** Max pooling provides:

- **Translation invariance:** Don't care about exact location of features
- **Dimensionality reduction:**  $28 \times 28 \rightarrow 14 \times 14$  with  $2 \times 2$  pooling
- **Feature detection:** Returns max value = "feature detected somewhere in region"

## 7 Complete CNN Architecture: Digit Recognition

### 7.1 LeNet-style Architecture for MNIST



### 7.2 Architecture Details

#### Layer-by-layer breakdown:

1. **Input Layer:**  $28 \times 28$  grayscale image (MNIST digit)
2. **Convolution 1:**
  - Filter size:  $5 \times 5$
  - Number of filters: 4
  - Output: 4 feature maps of size  $28 \times 28$
  - Detects low-level features (edges, curves)
3. **Max Pooling 1:**
  - Pool size:  $2 \times 2$
  - Output: 4 feature maps of size  $14 \times 14$
  - Provides local translation invariance
4. **Convolution 2:**
  - Filter size:  $5 \times 5$
  - Number of filters: 12
  - Output: 12 feature maps of size  $10 \times 10$
  - Detects higher-level features (combinations of edges)
5. **Max Pooling 2:**
  - Pool size:  $2 \times 2$
  - Output: 12 feature maps of size  $5 \times 5$
  - Further invariance and dimensionality reduction
6. **Flatten:** Convert  $12 \times 5 \times 5 = 300$  features into a vector
7. **Fully Connected Layer:** Dense hidden layer for classification
8. **Output Layer:** 10 units (digits 0-9) with softmax activation

## 8 Key Benefits of CNNs

### Advantages of CNNs

1. **Sparse Connectivity:** Each output connects to a small local region
  - Fully connected:  $n_{input} \times n_{output}$  connections
  - CNN:  $k \times k \times n_{output}$  connections (where  $k$  is filter size)
2. **Parameter Sharing:** Same filter weights used across spatial locations
  - Fully connected:  $n_{input} \times n_{output}$  parameters
  - CNN:  $k \times k \times n_{filters}$  parameters
3. **Translation Invariance:** Through pooling layers
  - Detects features regardless of position
  - Handles small spatial variations
4. **Automatic Feature Learning:** No manual feature engineering
  - Weights learned via backpropagation
  - Network discovers optimal features for task
5. **Variable Input Lengths:** Can handle inputs of varying sizes (with appropriate architecture adjustments)

## 9 Convolutional Layer Hyperparameters

When defining a convolutional layer, specify these parameters:

### 9.1 Number of Filters

- Determines how many feature maps are produced
- Example: 32, 64, 128 filters common in practice
- More filters = more features detected

### 9.2 Kernel Size (Filter Size)

- Width and height of the convolutional window
- Common sizes:  $3 \times 3$ ,  $5 \times 5$ ,  $7 \times 7$
- Larger kernels detect larger features

### 9.3 Stride

- How many positions to shift the filter
- Stride = 1: shift by 1 pixel at a time
- Stride = 2: shift by 2 pixels (results in smaller output)
- Larger stride = more downsampling



## 9.4 Padding

- **Valid padding:** No padding, output smaller than input
- **Same padding:** Add zeros around input so output size = input size

## 9.5 Output Size Calculation

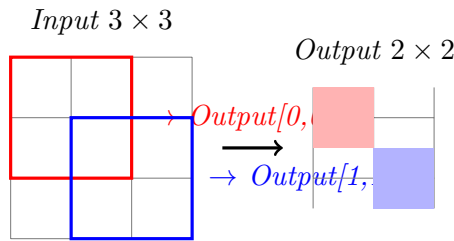
For a convolutional layer:

$$\text{output\_size} = \left\lfloor \frac{\text{input\_size} - \text{kernel\_size} + 2 \times \text{padding}}{\text{stride}} \right\rfloor + 1 \quad (9)$$

## 10 Hyperparameter Examples

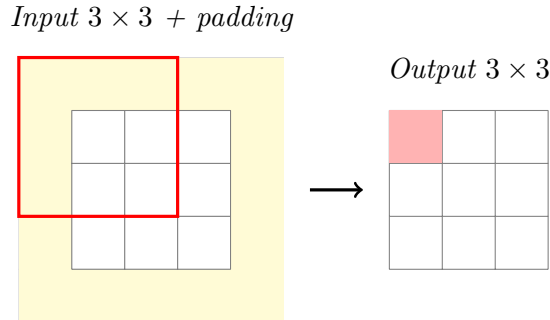
**Example 2** (Configuration 1). *Parameters:* Input  $3 \times 3$ , Kernel  $2 \times 2$ , Stride  $(1, 1)$ , Padding = “valid”

**Result:** Output  $2 \times 2$



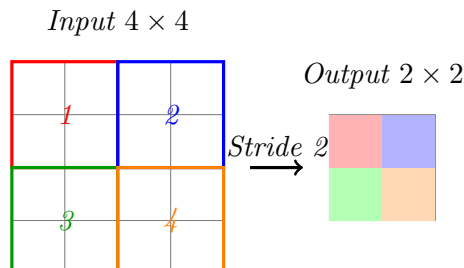
**Example 3** (Configuration 2). *Parameters:* Input  $3 \times 3$ , Kernel  $3 \times 3$ , Stride  $(1, 1)$ , Padding = “same”

**Result:** Output  $3 \times 3$  (padding adds zeros around input)



**Example 4** (Configuration 3). *Parameters:* Input  $4 \times 4$ , Kernel  $2 \times 2$ , Stride  $(2, 2)$ , Padding = “valid”

**Result:** Output  $2 \times 2$  (stride of 2 causes more downsampling)



## 11 Training CNNs

### 11.1 Backpropagation with Weight Sharing

CNNs are trained using standard backpropagation with gradient descent, despite weight sharing:

- **Forward pass:** Compute outputs layer by layer
- **Backward pass:** Compute gradients using chain rule
- **Weight sharing:** Same weight appears multiple times in computation graph
- **Gradient accumulation:** Gradients from all instances are summed

**Automatic Differentiation:** Modern frameworks (TensorFlow, PyTorch, Keras) handle shared weights automatically when computing gradients.

### 11.2 Optimization Algorithms

All standard optimization algorithms work with CNNs:

- Stochastic Gradient Descent (SGD)
- SGD with Momentum
- Adam
- RMSprop
- AdaGrad

## 12 Important Insights

### Key Takeaways

#### 1. Automatic Feature Learning

- CNNs learn features automatically through backpropagation
- No need for manual feature engineering
- Features may not be easily interpretable by humans
- Data-driven features often outperform hand-crafted features

#### 2. Why Weight Sharing Works

- Feature detection (e.g., edge) doesn't depend on location
- Same filter applied everywhere detects same feature anywhere
- Example: Detecting phonemes in audio at different times
- Example: Detecting faces in different parts of an image

#### 3. Feature Hierarchy

- Early layers: Low-level features (edges, corners, textures)
- Middle layers: Mid-level features (parts, simple patterns)
- Later layers: High-level features (objects, faces, complex patterns)
- Network builds hierarchical representation automatically

#### 4. Random Initialization is Critical

- Different feature maps must learn different features
- Random initialization ensures filters diverge during training
- Without randomization, all filters might learn same features

## 13 Historical Context & Applications

**Biological Inspiration:** CNN architectures are inspired by the organization of the animal visual cortex, particularly the work on simple and complex cells by Hubel and Wiesel.

### Modern Applications:

- Image classification (ImageNet, MNIST)
- Object detection (YOLO, R-CNN)
- Face recognition
- Medical image analysis
- Autonomous vehicles
- Natural language processing (with 1D convolutions)

- Speech recognition
- Video analysis

## 14 Summary

Convolutional Neural Networks revolutionized computer vision by:

1. Exploiting spatial structure in data through local connectivity
2. Reducing parameters dramatically through weight sharing
3. Learning hierarchical features automatically
4. Achieving translation invariance through pooling
5. Scaling to high-dimensional inputs efficiently

The key innovation is treating neural network layers as learnable convolutional filters, allowing the network to discover optimal features for the task at hand rather than relying on hand-engineered features.